Project work part 1

- Author: Mehdi Jamali

Links

- GitHub:
- Streamlit:
- Video:

Declaration of AI usage

The following AI tools were used during the development of this project:

- **GitHub Copilot:**
  - Used for code completion, suggestions, and feedback during development.
- **Gemini:**
  - Used for coding assistance and for finding relevant functions and methods.
- **NotebookLM (Gemini):**
  - Used as a research assistant to answer questions and explain topics related to the project.

The suggestions were reviewed, adapted, and tested before they were used in the project.

The AI tools were used within the guidelines of "K3 - Full use of AI" at NMBU.

Work log

I worked on Part 1 of the IND320 project using both Jupyter Notebook and Streamlit. I started by loading the `reservoirs.csv` dataset with Pandas. The dataset contained Norwegian column names, so I renamed them to clear English names, including date, area type, area number, year, week, filling level, capacity, filled volume, and change from the previous week. I also converted the date column to the correct datetime format so it could be used for sorting and visualizations.

In the Jupyter Notebook, I explored the structure and contents of the dataset. I used Pandas to inspect the data and prepared it for analysis. I worked on displaying the data clearly and creating visualizations of the different variables. Because the columns have different units and scales, I considered how the variables should be plotted separately and together so that the results are easier to understand.

I also developed a Streamlit application for presenting the project. The application has a main page with information about the dataset and a sidebar for navigation. I created separate pages for the home page, data table, interactive plotting, and an additional page for future analysis. The data table page loads the reservoir data and allows users to filter the data by reservoir area and year. The interactive plotting page allows users to select an area, year, and measurement, and then view changes over time.

The project is stored in a public GitHub repository and deployed as a Streamlit application. I

learned how to connect the data analysis in the notebook with the interactive app format in Streamlit. I also gained experience with caching data, creating user controls, and publishing a project online.

I cooperated with Ole Andreas Grøtting at Bikuben. I also communicated through Messenger with Ole Andreas Grøtting, Daniel Tsan, and Oliver Madsen. We discussed the project requirements, coding problems, possible solutions, and the structure of the application.

I used GitHub Copilot, Gemini, and NotebookLM as support during development. These tools helped with code suggestions, explanations, and ideas for solving programming problems. I reviewed, adapted, and tested the suggestions before using them in the project.

Library imports

We start by importing the necessary libraries for the project:

```
import pandas as pd
import numpy as np
```

Next we read the dataframe and translate the headers to english:

```
# Load the reservoir observations from the data folder.
df = pd.read_csv("Data/reservoirs.csv")

# Rename the Norwegian headers to clear English.
df = df.rename(
    columns={
        "dato_Id": "date",
        "omrType": "area_type",
        "omrnr": "area_number",
        "iso_aar": "year",
        "iso_uke": "week",
        "fyllingsgrad": "filling_level",
        "kapasitet_TWh": "capacity_TWh",
        "fylling_TWh": "filled_TWh",
        "neste_Publiseringsdato": "next_publication_date",
        "fyllingsgrad_forrige_uke": "previous_week_filling_level",
        "endring_fyllingsgrad": "change_in_filling_level",
    }
)

# Convert the date column so that it can be sorted and plotted correctly.
df["date"] = pd.to_datetime(df["date"])
```

Data overview

The renamed dataset is displayed below to verify that the data was loaded and prepared correctly.

```
# Display the first five rows to verify that the data was imported and renamed correctly.
df.head(5)
```

Reservoir variable plots

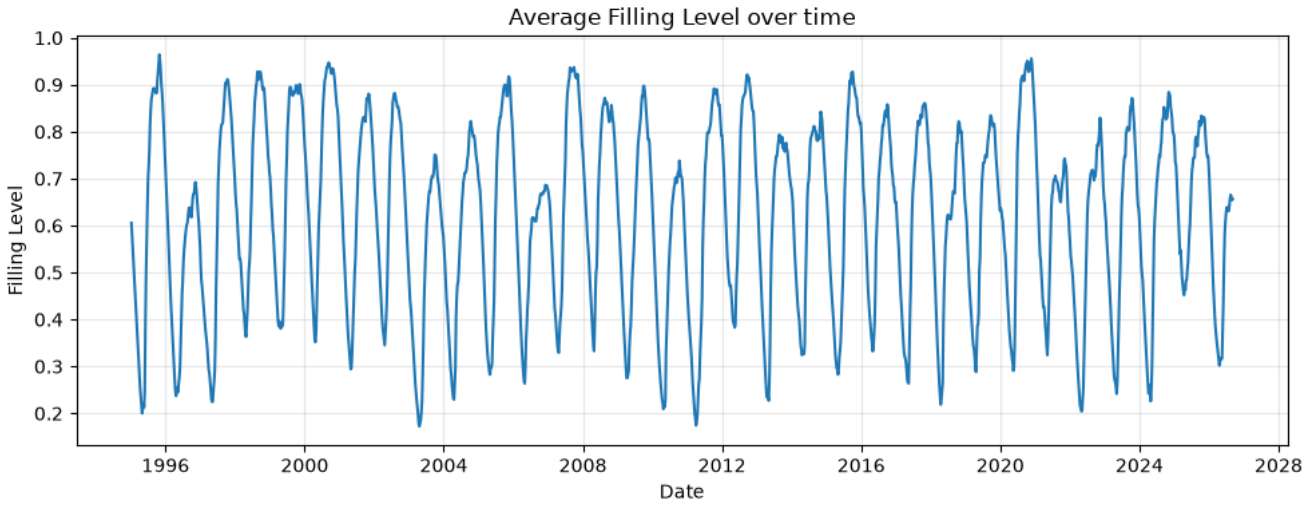
The numeric measurements are plotted separately and then compared in one standardized plot.

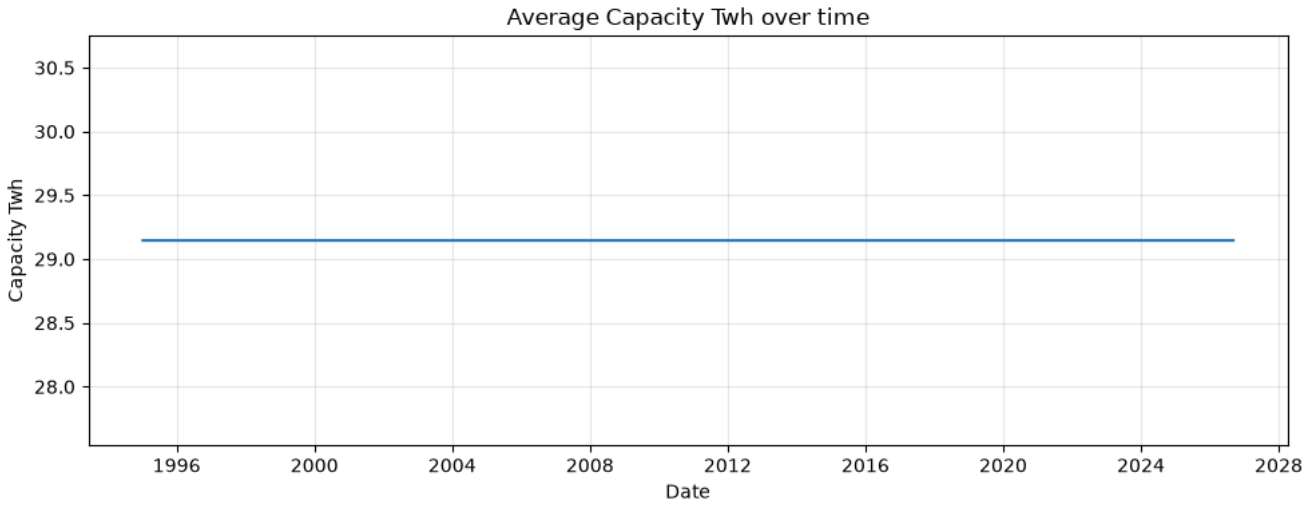
```
import matplotlib.pyplot as plt

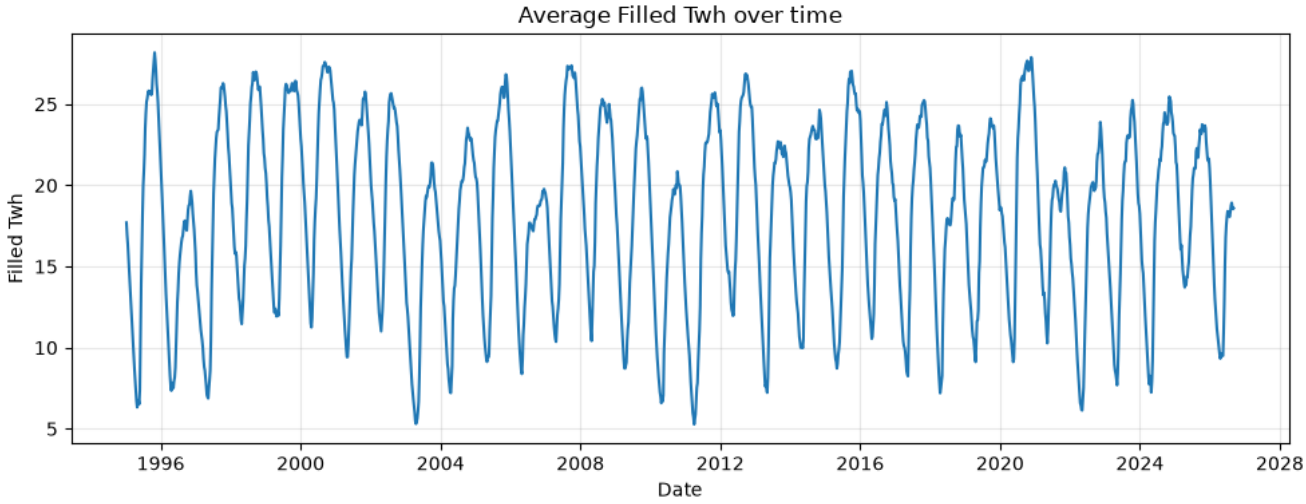
# Select measurements that are useful to compare over time.
# Identifier columns such as year and week are intentionally excluded.
plot_columns = [
    "filling_level",
    "capacity_TWh",
    "filled_TWh",
    "previous_week_filling_level",
    "change_in_filling_level",
]

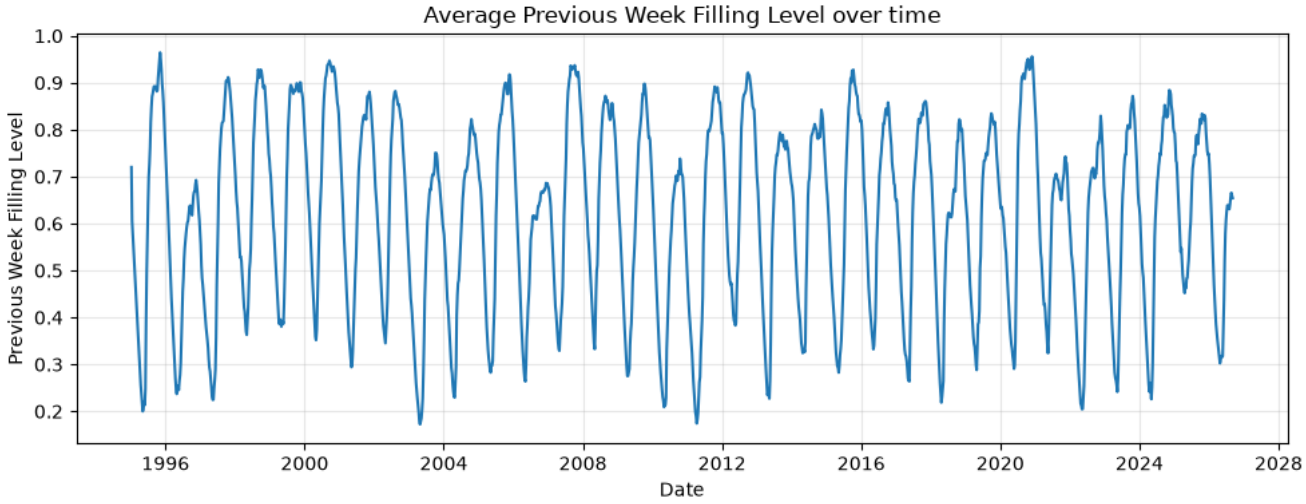
# Several reservoir areas can have observations on the same date.
# Calculate the mean for each date so the plots show one clear national trend.
plot_data = df.groupby("date", as_index=False)[plot_columns].mean()

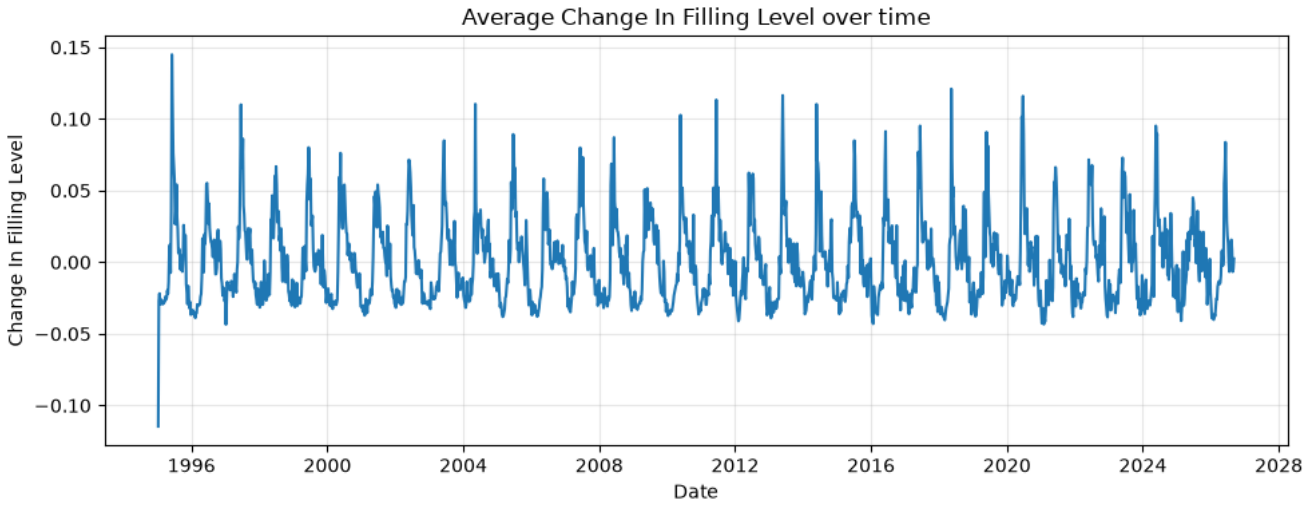
# Plot each measurement separately because the variables use different units.
for column in plot_columns:
    plt.figure(figsize=(10, 4))
    plt.plot(plot_data["date"], plot_data[column], linewidth=1.5)
    plt.title(f"Average {column.replace('_', ' ').title()} over time")
    plt.xlabel("Date")
    plt.ylabel(column.replace("_", " ").title())
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
```




Average Capacity Twh over time
Capacity Twh
30.5
30.0
29.5
29.0
28.5
28.0
1996
2000
2004
2008
2012
2016
2020
2024
2028
Date


Average Filled Twh over time
Filled Twh
25
20
15
10
5
1996
2000
2004
2008
2012
2016
2020
2024
2028
Date


Average Previous Week Filling Level over time
Previous Week Filling Level
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
1996
2000
2004
2008
2012
2016
2020
2024
2028
Date




```
# Standardize the daily averages so variables with different units can be compared.
# Each series is converted to standard deviations from its own mean.
standardized_data = plot_data[plot_columns].apply(
    lambda column: (column - column.mean()) / column.std()
)
standardized_data["date"] = plot_data["date"]

# Plot all standardized measurements together on a common scale.
ax = standardized_data.plot(
    x="date",
    figsize=(12, 6),
    linewidth=1.5,
    title="Standardized average reservoir measurements over time",
)
ax.set_xlabel("Date")
ax.set_ylabel("Standard deviations from the mean")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

